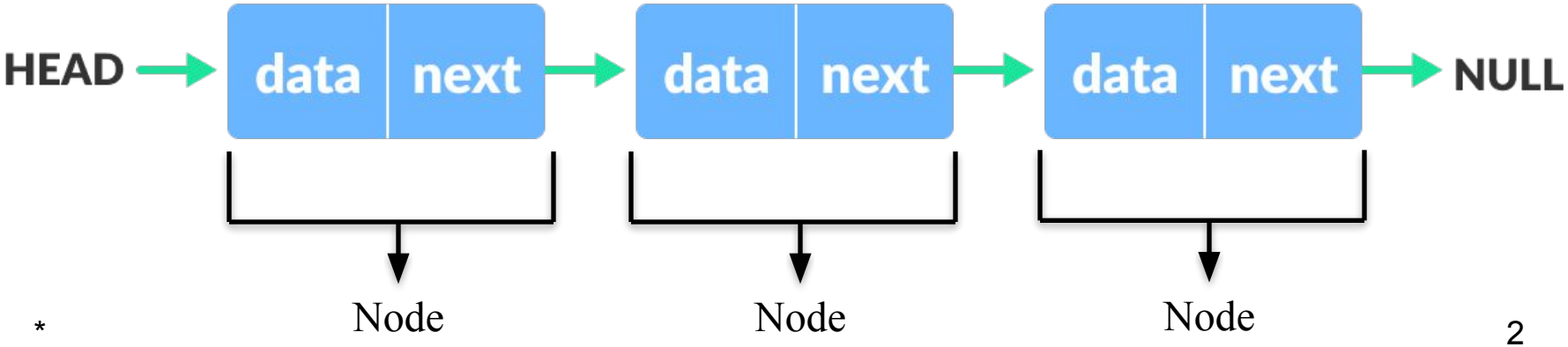
Linked List

Linked List

- A linked list is an ordered collection of finite, homogeneous data elements called nodes where the linear order is maintained by means of links or pointers.
- Each element is represented by a node.
- Each node is divided into two parts.

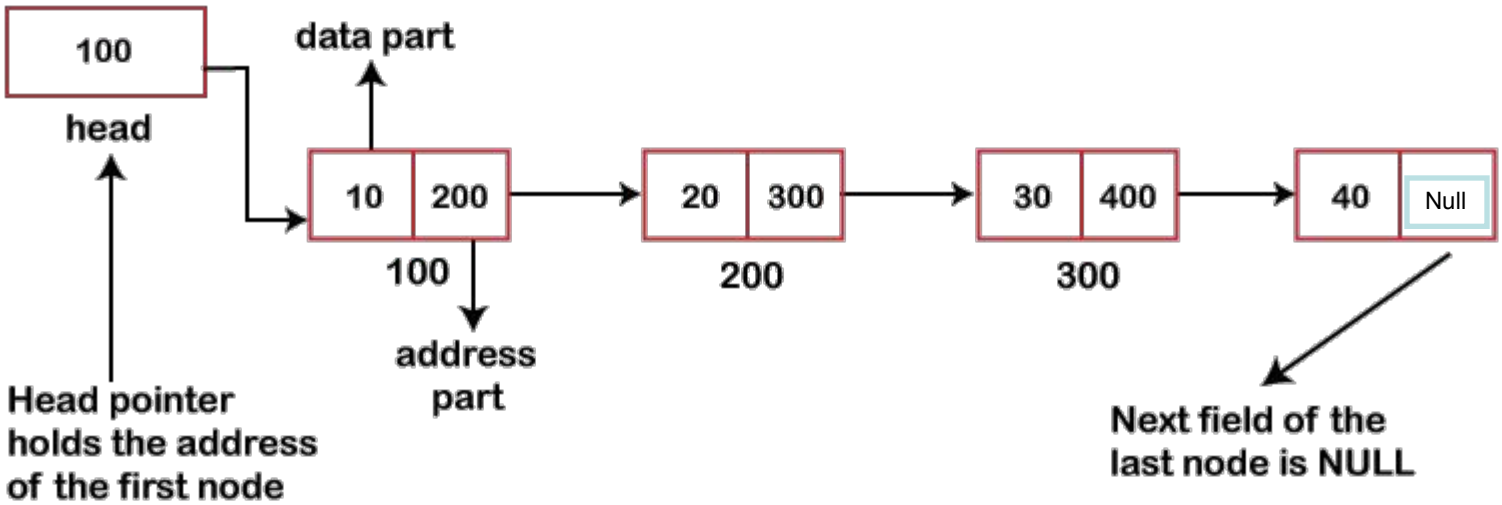
1. **Data** part contains the information of element.
2. **Link** part contains the address of the **next** node in the link.



Linked List

- Linked list has a list pointer variable, **Head** or **Start**, containing the address of the 1st node in the list.
- If Start contains NULL value, it means the list is empty.
- The address field of last node in the list contains NULL representing invalid address/ no other node after it.

Example: The following figure shows a linked list having 4 nodes.



Memory Representation of Linked List

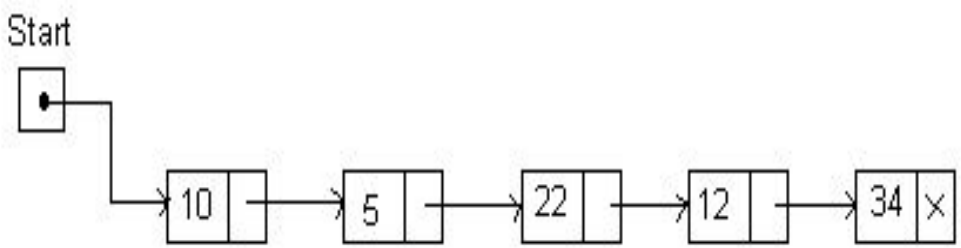
There are two ways to represent a linked list in memory:

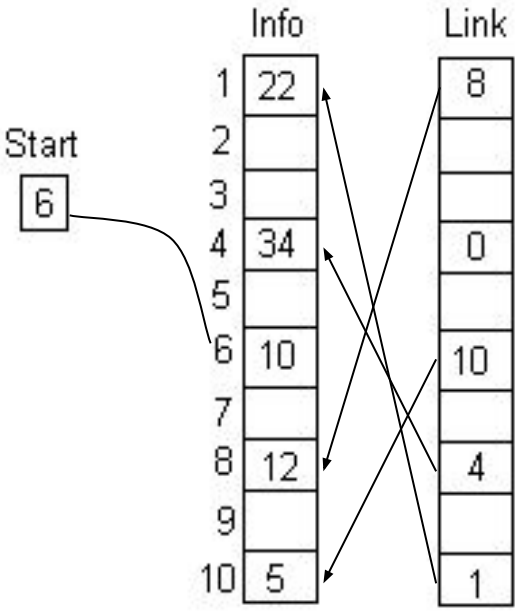
1. Static representation using array
2. Dynamic representation using free pool of storage

Static representation using array: A linked list can be maintained in memory using two linear arrays.

1. Info
2. Link

- Info[K] represents the information part of a node in the list.
- Link[K] represents the next pointer field of a node in the list.
- The beginning of the list is denoted by the variable Start.





* Figure: Linked list & its memory representation

Dynamic representation

- Using the free pool of storage.

- A memory bank (which is a collection of free memory spaces)
- A memory manager (a program).

During the creation of a linked list, whenever a node is required the request is placed to the memory manager; the memory manager will then search the memory bank for the block requested and, if found, grants the desired block to the caller.

Advantages of Linked lists over arrays:

- Elements can be inserted into linked lists indefinitely, while an array will eventually either fill up or need to be resized, an expensive operation that may not even be possible if memory is fragmented.
- An array from which many elements are removed may become wastefully empty or need to be made smaller.
- Further memory savings can be achieved, in certain cases, by sharing the same "tail" of elements among two or more lists — that is, the lists end in the same sequence of elements. In this way, one can add new elements to the front of the list while keeping a reference to both the new and the old versions — a simple example of a persistent data structure.

Disadvantages of Linked lists over arrays:

- Arrays allow random access, while linked lists allow only sequential access to the elements.
- Singly-linked lists can only be traversed in one direction. This makes linked lists unsuitable for applications where it is useful to look up an element by its index quickly, such as heapsort.
- Another disadvantage of linked lists is the extra storage needed for references.

Array vs Linked list

ARRAY	LINKED LISTS
1. Arrays are stored in contiguous location.	1. Linked lists are not stored in contiguous location.
2. Fixed in size.	2. Dynamic in size.
3. Memory is allocated at compile time.	3. Memory is allocated at run time.
4. Uses less memory than linked lists.	4. Uses more memory because it stores both data and the address of next node.
5. Elements can be accessed easily.	5. Element accessing requires the traversal of whole linked list.
6. Insertion and deletion operation takes time.	6. Insertion and deletion operation is faster.

ArrayList Vs LinkedList

ArrayList	LinkedList
1) ArrayList internally uses a **dynamic array** to store the elements.	Linked List internally uses a **doubly linked list** to store the elements.
2) Manipulation with ArrayList is **slow** because it internally uses an array. If any element is removed from the array, all the other elements are shifted in memory.	Manipulation with LinkedList is **faster** than ArrayList because it uses a doubly linked list, so no bit shifting is required in memory.
3) An ArrayList class can **act as a list** only because it implements List only.	LinkedList class can **act as a list and queue** both because it implements List and Deque interfaces.
4) ArrayList is **better for storing and accessing** data.	LinkedList is **better for manipulating** data.
5) The memory location for the elements of an ArrayList is contiguous.	The location for the elements of a linked list is not contagious.
6) Generally, when an ArrayList is initialized, a default capacity of 10 is assigned to the ArrayList.	There is no case of default capacity in a Linked List. In Linked List, an empty list is created when a Linked List is initialized.
7) To be precise, an ArrayList is a resizable array.	LinkedList implements the doubly linked list of the list interface.

Types of Linked list

- **Singly-linked list -** A node in the singly linked list consists of two parts: data part and link part. Data part of the node stores actual information that is to be represented by the node, while the link part of the node stores the address of its immediate successor.
- **Doubly linked list –** A node contains a pointer to the previous as well as the next node in the sequence. Therefore, in a doubly-linked list, a node consists of three parts: node data, pointer to the next node in sequence (next pointer), and pointer to the previous node (previous pointer).
- **Circular singly linked list -** The last node of the list contains a pointer to the first node of the list.
- **Circular doubly linked list -** A node contains pointers to its previous node as well as the next node. Circular doubly linked list does not contain NULL in any of the nodes. The last node of the list contains the address of the first node of the list. The first node of the list also contains the address of the last node in its previous pointer.

Terminologies

- **Structures** are a way to group several related variables into one place. Each variable in the structure is known as a member of the structure. Unlike an array, a structure can contain many different data types (int, float, char, etc.).
- Create a Structure

```
struct MyStructure {   // Structure declaration
  int myNum;          // Member (int variable)
  char myLetter;      // Member (char variable)
}; // End the structure with a semicolon
```

- Use the struct keyword inside the main() method, followed by the name of the structure and then the name of the structure variable:
- Create a struct variable with the name "s1":

```
struct myStructure {
  int myNum;
  char myLetter;
};
int main() {
  struct myStructure s1;
  return 0;
}
```

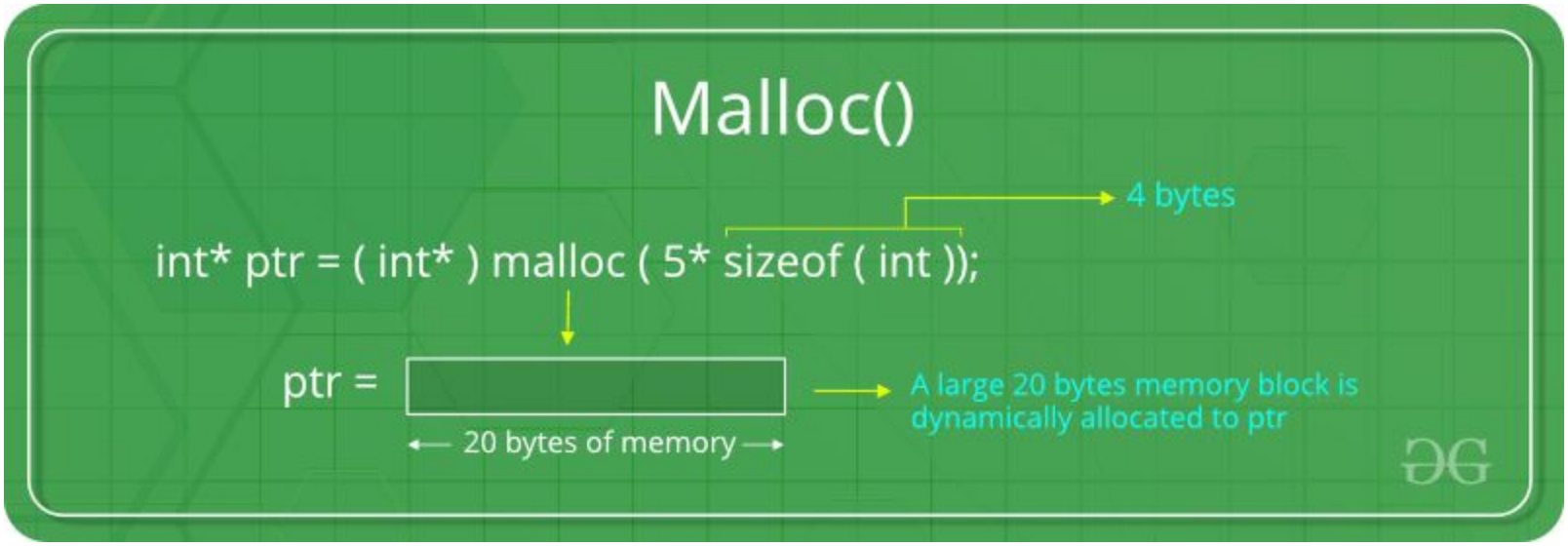
Terminologies

Dynamic Memory allocation:

The **"malloc"** or **"memory allocation"** method in C is used to dynamically allocate a single large block of memory with the specified size. It returns a pointer of type void which can be cast into a pointer of any form. malloc() returns NULL if it cannot fulfill the request.

Syntax:

ptr = (cast-type*) malloc(byte-size)



*

Terminologies

sizeof():

It is a compile-time unary operator and used to compute the size of its operand. It returns the size of a variable. It can be applied to any data type, float type, pointer type variables.

Syntax:

```
sizeof(Expression);
```

where ‘Expression’ can be a data type or a variable of any type.

Terminologies

free() method

“free” method in C is used to dynamically deallocate the memory. The memory allocated using functions malloc() is not deallocated on their own. Hence the free() method is used, whenever the dynamic memory allocation takes place. It helps to reduce wastage of memory by freeing it.

Syntax:

```
free(ptr);
```

Commonly used operations on Singly Linked List

- Insertion: The insertion operation can be performed in three ways.
  - Inserting At the Beginning of the list
  - Inserting At End of the list
  - Inserting At Specific location in the list
- Deletion: The deletion operation can be performed in three ways.
  - Deleting from the Beginning of the list
  - Deleting from the End of the list
  - Deleting a Specific Node
- Search: It is a process of determining and retrieving a specific node either from the front, the end or anywhere in the list.
- Display: This process displays the elements of a Single-linked list.

Representation of Linked List

- Each node consists:
  - A data item
  - An address of another node
- We wrap both the data item and the next node reference in a struct as:

```
struct node
{
  int data;
  struct node *next;
};
```

Creation of Nodes:

- Create a new struct node and allocate memory to it.
- Add its data value
- Point its next pointer to the struct node containing next node's value

Creating A Linked List

Here we will describe how can a linked list be implemented.

Create_List (Info, Link, Item, Start)

1. Set Info[New] := Item and Link[New] := NULL /…New represents a new node
2. If Start = NULL /…List is empty.
3. then Set Start := New and goto step 9.
4. Set Save:=Start and PTR := Link[Start] /…List is not empty
5. While PTR ≠ NULL repeat steps 6 and 7
6. Set Save :=PTR
7. Set PTR := Link[PTR].
8. Set Link[Save]:=New.
9. go to step 1 /…For another new item

A linked list with 3 nodes

```
/* Initialize nodes */
struct node *head;
struct node *one = NULL;
struct node *two = NULL;
struct node *three = NULL;

/* Allocate memory */
one = malloc(sizeof(struct node));
two = malloc(sizeof(struct node));
three = malloc(sizeof(struct node));
```

```
/* Assign data values */
one->data = 1;
two->data = 2;
three->data=3;

/* Connect nodes */
one->next = two;
two->next = three;
three->next = NULL;

/* Save address of first node in head */
head = one;
```

Insertion (1)

Add a node at the front:

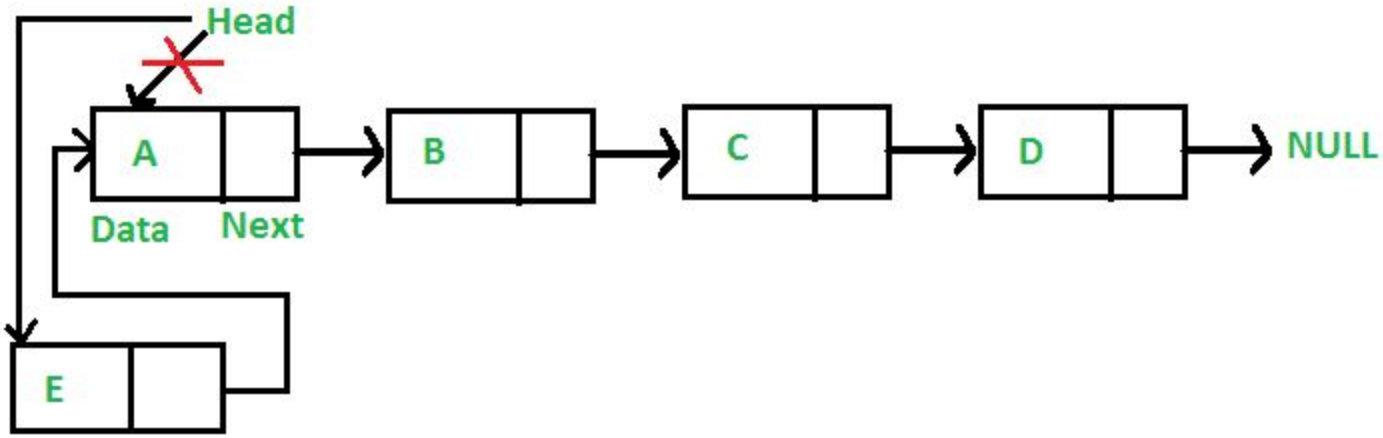
Approach: The new node is added before the head of the given Linked List. And newly added node becomes the new head of the Linked List.

Example:

Given Linked List is, A->B->C->D

Add an item E at the front,

Then the Linked List becomes E-> A->B->C->D

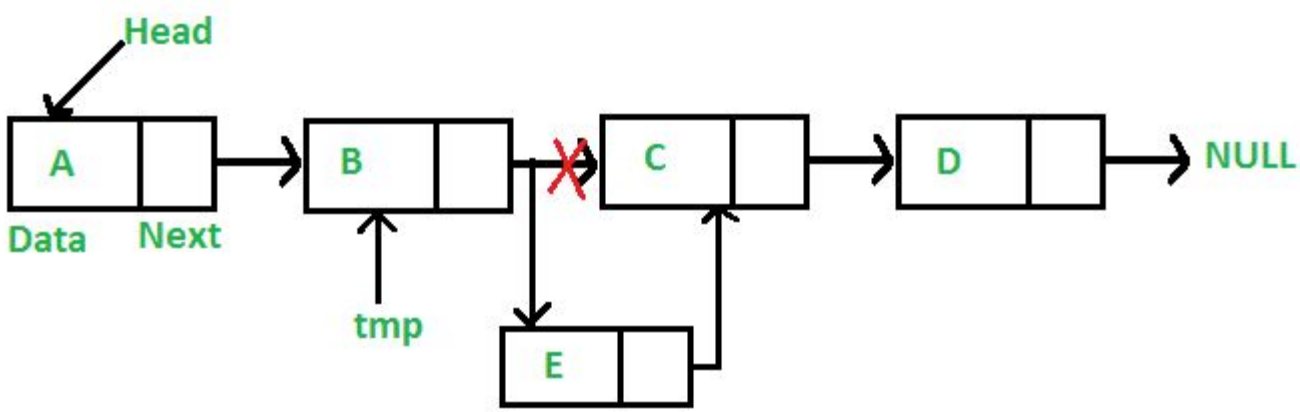


Insertion (2)

Add a node after a given node:

Approach: We are given a pointer to a node, and the new node is inserted after the given node.

- Firstly, check if the given previous node is NULL or not.
- Then, allocate a new node and
- Assign the data to the new node
- And then make the next of new node as the next of previous node.
- Finally, move the next of the previous node as a new node.



Insertion (3)

Add a node at the end:

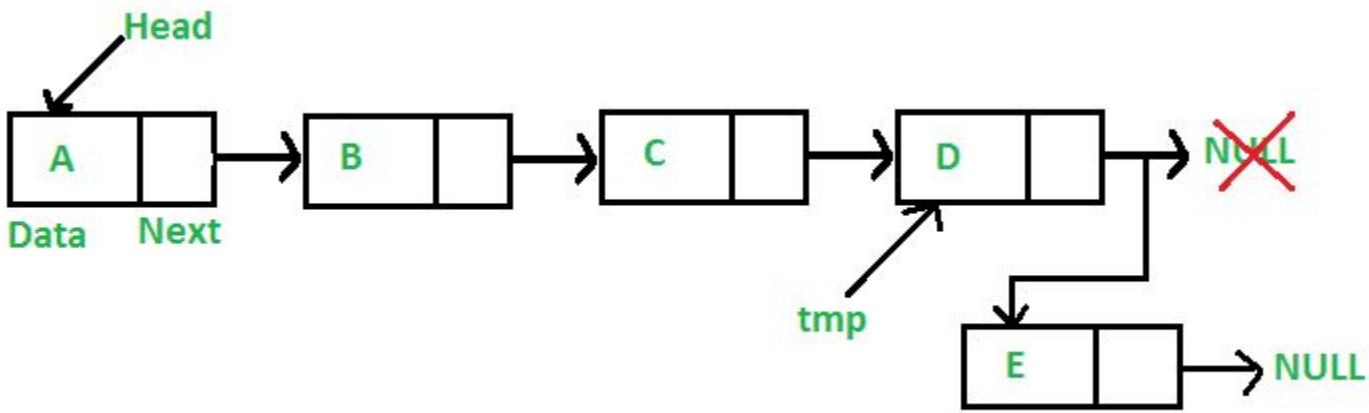
•The new node is added after the last node of the given Linked List.

Example:

Given Linked List is A->B->C->D

Add an item E at the end,

Then the Linked List becomes A->B->C->D ->E.



Deletion

To delete a node from the linked list, we need to do the following steps:

- Find the **previous node** of the node to be deleted.
- Change the **next** of the previous node.
- Free memory for the node to be deleted.

Deletion of an element in a list from:

- Beginning
- End
- Middle

Deletion (1)

1. If the head node has the given key,

make the head node points to the second node and free its memory.

2. Otherwise,

From the current node, check whether the next node has the given key

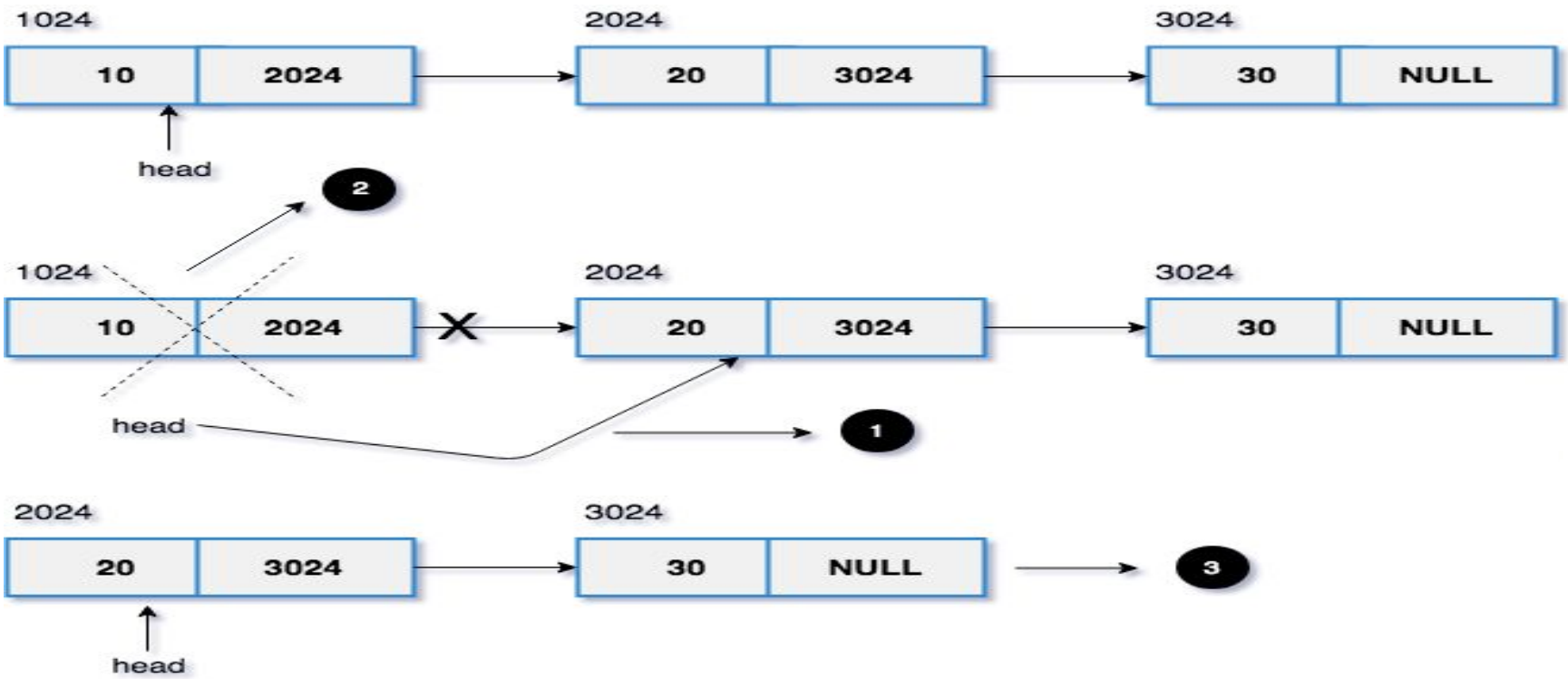
if yes, make the current->next = current->next->next and free the memory.

else, update the current node to the next and do the above process (from step 2) till the last node.

Deletion (1)

Deletion from head:

1. Make the head points to the next node.
2. Free the head node's memory.
3. Finally, the new linked list.

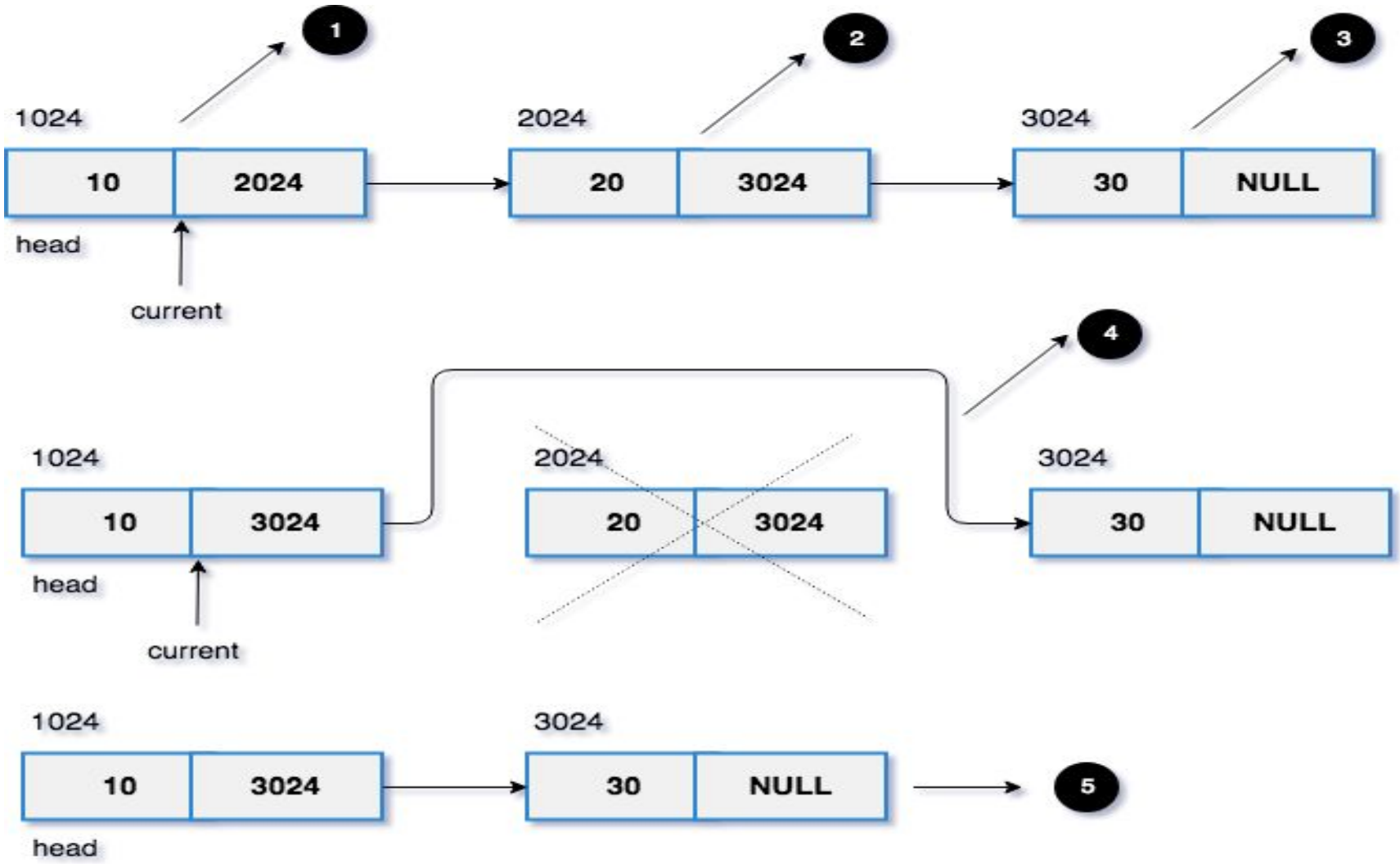


Deletion (2)

Deletion from other nodes (Non-Head/from middle)

1. Make the current node points to the head node. (current => data = 10).
2. current => next. (current=>next=>data = 20).
3. current => next => next. (current=>next=>next=>data = 30).
4. We have to remove the node 20. Since current => next = 20 we can remove the node by setting current => next = current =>next => next. And then free the node.
5. Finally, the new linked list.

Deletion (2)


1
2
3
1024
10
2024
head
current
2024
20
3024
3024
30
NULL
4
1024
10
3024
head
current
2024
20
3024
3024
30
NULL
1024
10
3024
head
3024
30
NULL
5

Deletion (3)

Delete from End:

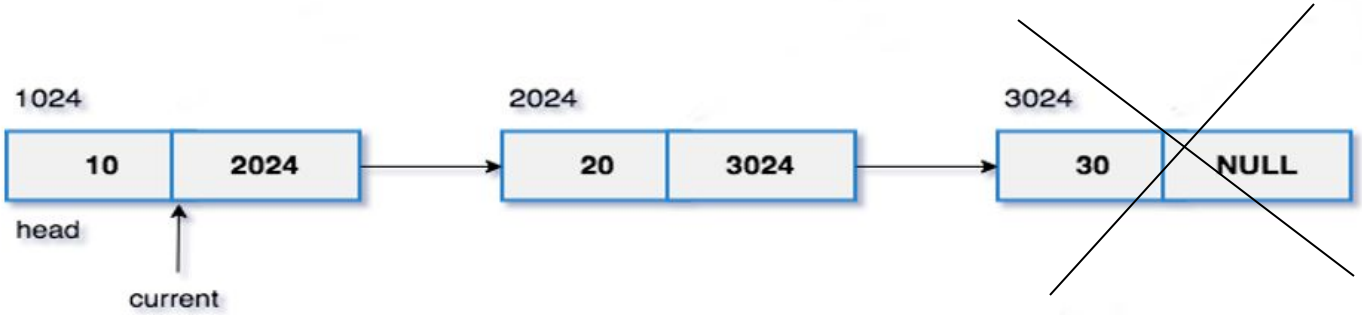
- Point head to the previous element i.e. last second element
- Change next pointer to null

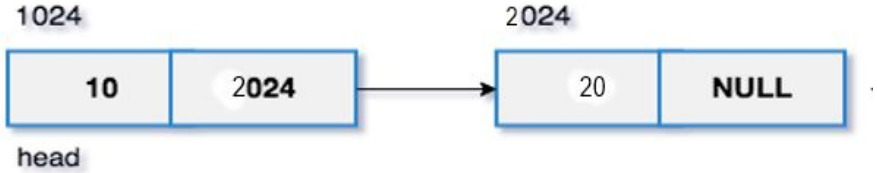
```
struct node *end = head;
struct node *prev = NULL;
while(end->next)
{
    prev = end;
    end = end->next;
}
prev->next = NULL;
```

- Make sure to free unused memory
  - free(end); or delete end;

Deletion (3)

Delete from End:





Traversing A Linked List

Traverse_List (List, Info, Link, Start)

1. Set PTR := Start
2. While PTR ≠ NULL repeat steps 3 and 4
3. Apply process to Info[PTR]
4. PTR := Link[PTR]
5. Return.

Counting The No. of Elements in a Linked List

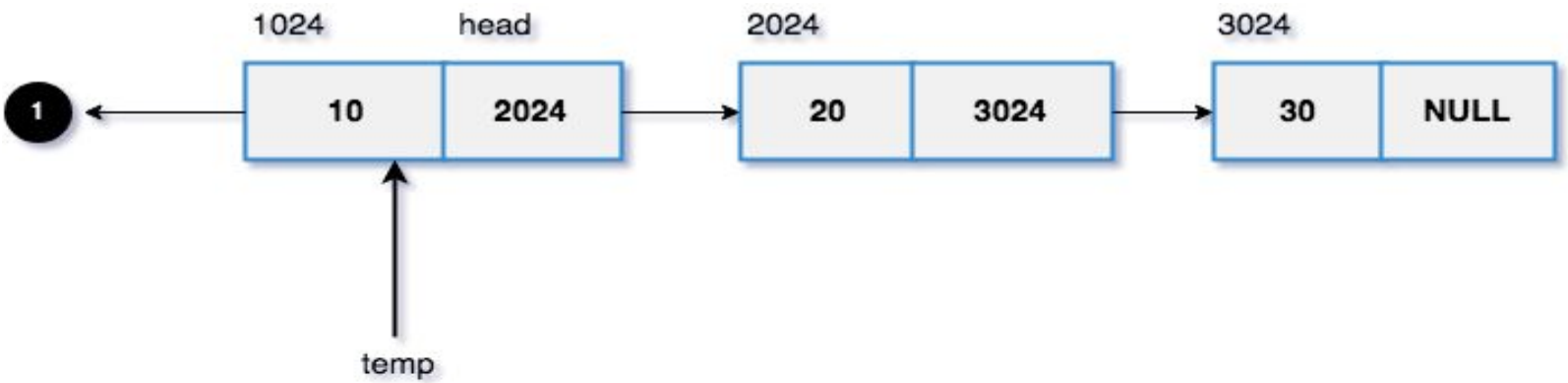
Count_Node_List (List, Info, Link, Start)

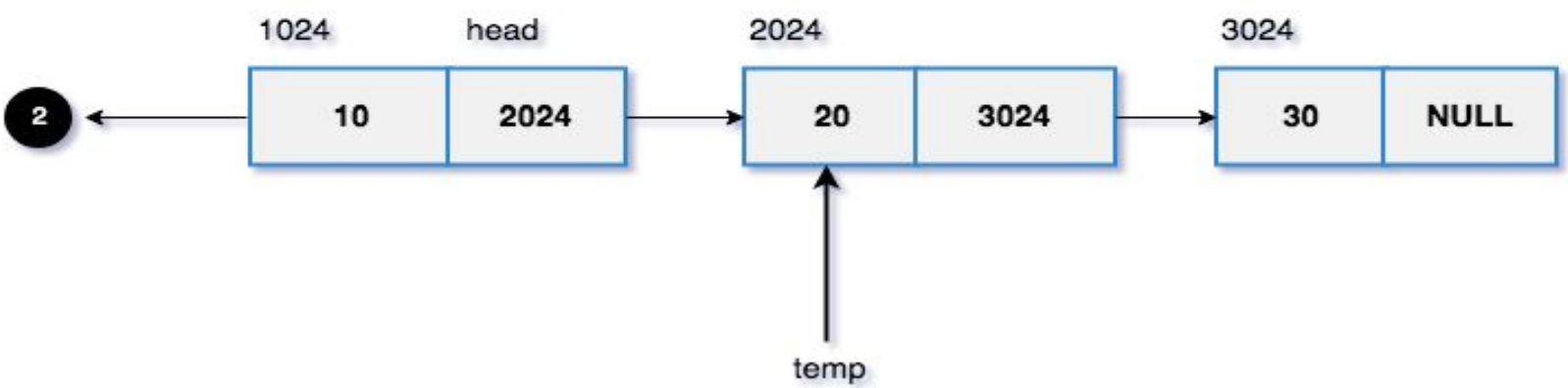
1. Set PTR := Start and Num := 0
2. While PTR ≠ NULL repeat steps 3 and 4
3. Num := Num + 1
4. PTR := Link[PTR]
5. Return.

*

Searching a specific node

1. Iterate the linked list using a loop.
2. If any node has the given key value, return 1.
3. If the program execution comes out of the loop (the given key is not present in the linked list), return -1.





Searching a Linked List

1. List is unsorted
2. List is sorted

Searching an Item from an Unsorted List

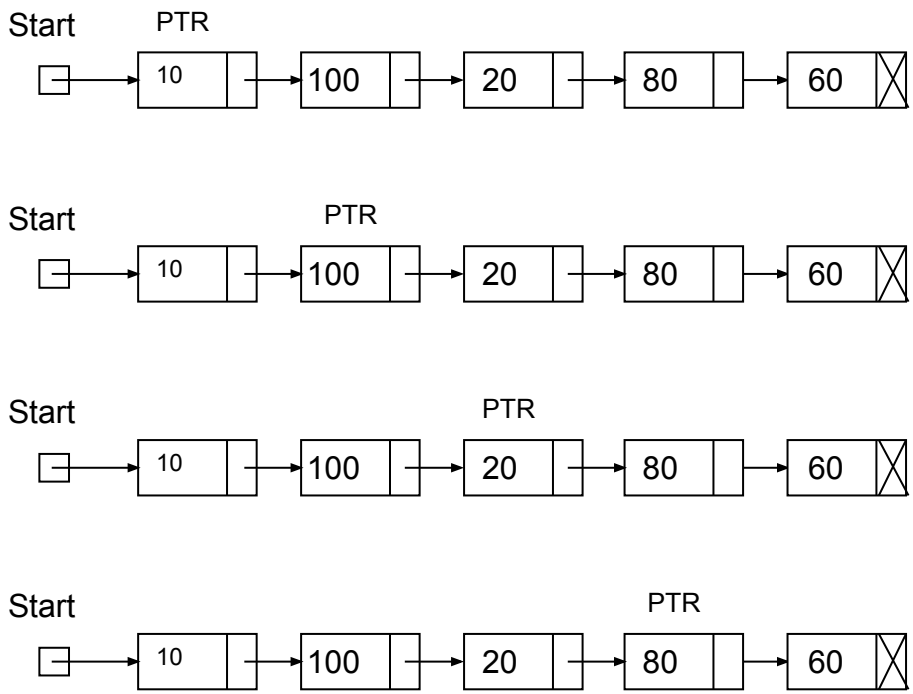
Search_UnsortedList (Info, Link, Start, Item, Loc)

1. Set PTR := Start and Loc := NULL
2. While PTR ≠ NULL do steps 3 to 5
3. If Item = Info[PTR] then
4. Loc := PTR , print: item found and return. /…Successful Search
5. PTR := Link[PTR]
6. If Loc = NULL then Print: 'Item not in the list.' /…Unsuccessful Search
7. Return

Example:

Suppose item 10 is in memory location 1.
item 100 is in memory location 2.
item 20 is in memory location 3.
item 80 is in memory location 4.
item 60 is in memory location 5.

Item to be searched is 80.



As Info[PTR] ≠ 80, PTR :=Link[PTR]

As Info[PTR] ≠ 80, PTR :=Link[PTR]

As Info[PTR] ≠ 80, PTR :=Link[PTR]

As Info[PTR] = 80, Loc := 4

Searching an Item from a Sorted List

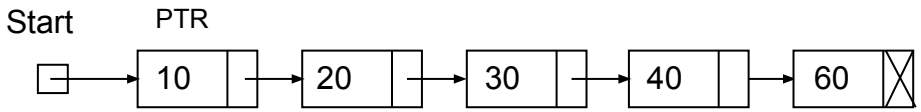
Sch_SortedList (Info, Link, Start, Item, Loc) / List is sorted in ascending order

```
1.  Set PTR := Start  and Loc := NULL
2.  While PTR ≠ NULL do steps 3 to 7
3.      if Item = Info[PTR]
4.          then Loc := PTR, print: Item found and return.    /…Successful Search
5.      else if Item > Info[PTR]
6.          then PTR := Link[PTR]
7.      else Exit.
8.  If Loc = NULL then Print: 'Item not in the list.'        /…Unsuccessful Search
9.  Return
```

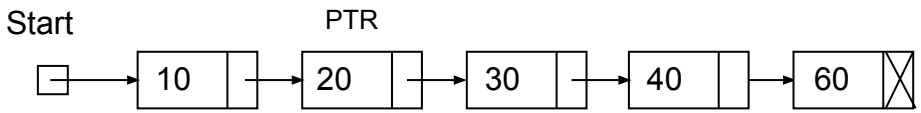
Example:

Suppose item 10 is in memory location 1.
item 20 is in memory location 2.
item 30 is in memory location 3.
item 40 is in memory location 4.
item 60 is in memory location 5.

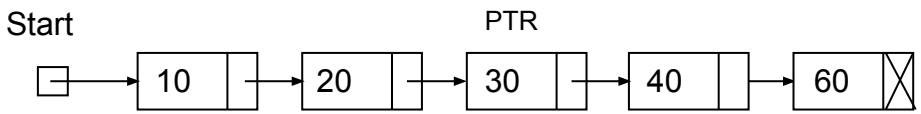
Item to be searched is 40.



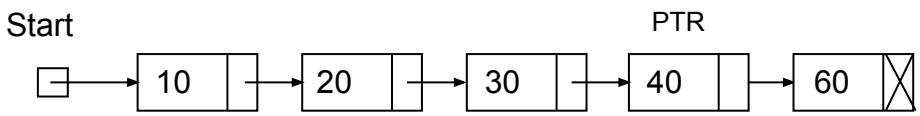
As Info[PTR] ≠ 40, PTR :=Link[PTR]



As Info[PTR] ≠ 40, PTR :=Link[PTR]



As Info[PTR] ≠ 40, PTR :=Link[PTR]

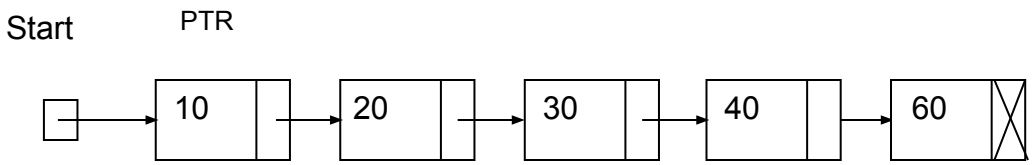


As Info[PTR] = 40, Loc := 4

Example:

Suppose item 10 is in memory location 1.
item 20 is in memory location 2.
item 30 is in memory location 3.
item 40 is in memory location 4.
item 60 is in memory location 5.

Item to be searched is 6.



As item 6 < Info[PTR] and the given list is already sorted in ascending order, so item can not be in the list.

So, Loc = NULL

END